① Let $f : R^n \to R$, then $f^* : R^n \to R$ is

Conjugate $f^*(y) = \sup_{x \in \mathrm{dom} f} (y^T x - f(x))$

Text BOYD (3.18)

conjugate $f^*(y)$ is max gap between linear fn yx and $f(x)$ → $\boxed{\text{geometrically}}$.

Following from the above definition

$$f(x) = \sum_{i=1}^{n} x_i \log x_i$$

Conjugate

$$f^*(y) = y^T x - f(x)$$

If f is differentiable, y, for $f^*(y)$ max occurs when $\frac{\partial}{\partial x_i}(y^T x - f(x)) = 0$

$\Rightarrow$ i.e. when $y = f'(x)$. $\boxed{y = \nabla f(x)}$

(1a)

Hence for $f(x) = \sum_{i=1}^{n} x_i \log x_i$

$y = f'(x) \Rightarrow \frac{\partial f(x)}{\partial x_i} \Rightarrow y_i = x_i \frac{1}{x_i} + \log x_i$.

$\log x_i = y_i - 1$

$x_i = e^{(y_i - 1)}$

1(a)

1(a)

writing the equation, by dropping subscripts (assume matrix form)

$\boxed{\text{Assume Matrix form}}$

$f^*(y) = y^T x - f(x)$

substitute $x = e^{(y-1)}$

$\Rightarrow \quad y^T e^{(y-1)} - e^{(y-1)} \log e^{(y-1)}$

$\Rightarrow \quad \cancel{y^T e^{(y-1)}} - \cancel{y e^{(y-1)}} + e^{(y-1)}$

$f^*(y) = e^{(y-1)}$

$\therefore$ The fenchel conjugate is

$$\boxed{f^*(y) = e^{(y-1)}}$$

$\boxed{\text{In subscript notation}}$

$$\boxed{f^*(y) = \sum_{i=1}^{n} e^{y_i - 1}}$$

1(b)

1(b) $$f(x) = -\sum_{i=1}^{n} \log(x_i)$$

Reference [BoYD 3.21] The problem is partially discussed (in section 3.3.1).

From definitions referenced in 1(a) (we can introduce subscript later)

The conjugate

$$f^*(y) = y^T x - f(x),$$

occurs at $y = f'(x)$.

1(b)

$$y = f'(x) \Rightarrow y = -\log x$$

$$\Rightarrow y = -\frac{1}{x} \quad \cancel{\Rightarrow -x = 2}$$

$$\Rightarrow x = -\frac{1}{y}$$

$$\therefore f^*(y) = y\left(-\frac{1}{y}\right) + \log\left(-\frac{1}{y}\right)$$

$$= -1 - \log(-y)$$

$$f^*(y) = -1 - \log(-y)$$

$$\Rightarrow \boxed{f^*(y) = \sum_{i=1}^{n} -1 - \log(-y_i)}$$

$\because \sum_{i=1}^{n} -1 = n$, $y_i < 0$ for log to be defined

$$\Rightarrow \boxed{f^*(y) = -n - \sum_{i=1}^{n} \log(-y_i)}$$

2 $f: \mathbb{R}^n \to \mathbb{R}$ show that $f^{**}(x) \leq f(x)$

we have the definition of conjugate

as $f^*(y) = \sup_x (y^T x - f(x))$

By D (3.3.1)

it follows that from the definition $\boxed{f(x) + f^*(y) \geq y^T x}$

$f \to f^*$ primal $(x) \to$ Dual (y)

$f^* \to f^{**}$ Dual $(y) \to$ primal (x)

let $g(y) = f^*(y)$; $g = f^*$

Applying conjugate to g

$$g^*(x) = \sup_y (x^T y - g(y))$$

$$f^{**}(x) = \sup_y (x^T y - f^*(y))$$

$$= \sup_y (y^T x - f^*(y))$$

we have from $f^*(y)$ definition - From Def above

$$\boxed{f(x) \geq y^T x - f^*(y)}$$

i.e. $y^T x - f^*(y)$ is upper bounded by $f(x)$

Q.2

(2)

$$f^{**}(x) = \sup_y \boxed{(y^T x - f^*(y))}$$

$\boxed{y^T x - f^*(y)}$ is upperbounded by $f(x)$

~~when~~ because $f(x) \geq y^T x - f^*(y)$

"supp" over all y is upper bonded by $f(x)$

$$\therefore \boxed{f^{**}(x) \leq f(x)}$$

~~supp~~

Q.3

(3) consider a step size of $\eta_t = 2^{-t}$

This is a geometric progression, with $r = 1/2$

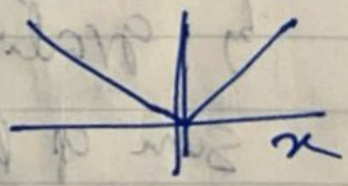
for $t=0$, to $t \to \infty$ summing this g.p $= \dfrac{1}{1-1/2}$

$= 2$

The series sum up to 2

i.e. sum of all the steps 2^{-t} converges to 2 for a function which has slope 2^{+1}

The limited range of steps, makes applying such a small step complex, leading to either vanishing gradients and/or poor convergence



Lets take $\boxed{f(x) = |x|}$

$\Delta x = -\nabla f(x)$, $\quad x := x + \eta \Delta x$.

lets start at $x = 100$, $\nabla f = \begin{cases} +1 & x > 0 \\ -1 & x < 0 \end{cases}$

Q.3

Example: 1

Take $f(x) = |x|$ as example

$$f(x) = |x| \quad , \quad \nabla f(x) = \begin{cases} 1 & x > 0 \\ -1 & x < 0 \\ [-1, +1] & x = 0 \end{cases}$$

Update: $x_{new} = x + t\,\Delta x \qquad \Delta x = -\nabla f(x)$

$x_0 = 10 \quad , 2^{-t}$

	x	step: η_t	$\Delta x = -\nabla f(x)$	$(\eta_t \cdot \Delta x)$	$x = x + \eta_t \Delta x$
$t=0$	10	$\frac{1}{2^0} = 1$	-1	-1	9
$t=1$	9	$\frac{1}{2^1} = 0.5$	-1	-0.5	8.5
$t=2$	8.5	$\frac{1}{2^2} = 0.25$	-1	-0.25	8.25
$t=3$	8.25	$1/2^3 = .125$	-1	-0.125	8.125

as it can be seen and seed

as $t \to \infty$ The $\sum_{t=0}^{\infty} \frac{1}{2^t} = \underline{\underline{2}}$ $\left(\text{G.P. sum } \frac{a}{1-r} \right)$

Hence the updated x will approximate to $\underline{8}$.

Even when $|x|$ is convex a step size of 2^{-t} does not converge to optimum

③ 3

Example 2 Q.3

Similarly take

$$\boxed{f(x) = \frac{x^2}{8}}$$

$$\nabla f(x) = \frac{2}{8}x = \frac{x}{4}$$

$$\boxed{x_{t+1} = x_t + \eta_t \Delta_t x \; ; \; \Delta x = -\nabla x}$$

$$x_{t+1} = x_t - 2^{-t}\left(\frac{x}{4}\right) = x_t - \frac{x_t}{(4)(2^t)}$$

But $\Delta x = -\nabla x$

$$x_{t_1} = x_{t_0} - \frac{x_{t_0}}{(4)\,2^t}$$

$$x_{t_2} = x_{t_1} - \frac{x_{t_1}}{(4 \cdot 2^t}$$

$$x_{t_3} = x_{t_2} - \frac{x_{t_2}}{(4)\,2^t}$$

$$\boxed{x_{t+1} = x_t\left(1 - \frac{1}{4 \cdot 2^t}\right)}$$

$$x_{t_1} = x_{t_0}\left(1 - \frac{1}{(4)(2)^t}\right) \quad t=0$$

$$x_{t_2} = a_{t_1}\left(1 - \frac{1}{(4)(2^t)}\right) \quad t=1$$

$$x_{t_3} = a_{t_2}\left(1 - \frac{1}{4 \cdot 2^t}\right) \quad t=2$$

~~Backward substitution form is~~

~~x_{t+1}~~

Q.3

(3) Example 2

(3)

Backward substitution Point is

$$x_{t+1} = x_0 \cdot \prod_{t=0}^{n} \left(1 - \frac{1}{(4)(2)^t}\right)$$

Such a series need not converge to zero for $f(x) = \frac{x^2}{8}$.

$1 - \frac{1}{(4)(2^t)} = 0.75$ for $t = 0$.

as $t \to \infty$ $\left(1 - \frac{1}{4 \cdot 2^t}\right) \simeq 1$

The product doesn't converge to zero

$t = 0$ $\quad 1 - \frac{1}{4 \cdot 2^t} = 0.75$

$t = 1$ $\quad 1 - \frac{1}{4 \cdot 2} = 0.875$

$t = 2$ $\quad 1 - \frac{1}{4 \cdot 4} = \frac{15}{16} = 0.937$

Rest of the terms will be greater than 0.937

~~$(0.75)(0.875)(0.937) \simeq 0.614$~~

~~x will converge to some value around~~

Q.3

(3) $(0.75)(0.875)(0.937) \simeq$ ~~0.987~~ 0.614

for $t=3$ $\left(1-\frac{1}{4\cdot 2^3}\right) \simeq 0.9687$

Hence The product approximately converges to ~~$x_0(0.61)(0.96)$~~ $(0.58)\,x_{t_0}$ after 4 iterations

as $\left(1-\frac{1}{(4)(2^t)}\right)$ approaches 1

x_{t+1} will not change a whole lot and will not converge to zero

Hence the gradient descent fails to approach optimal solution

4 $\min f(x) = \frac{1}{2} x^T \cdot Q x + q^T x,$

$Q > 0$ (Q : strictly +ve semidefin)

Q^{-1} Inverse.

4(a) $f(x) = \frac{1}{2} x^T Q x + q^T x$

$$\boxed{\nabla f(x) = Qx + q^T}.$$

Refer lecture 9.2

Equate Gradient to zero.

Taking $\nabla f(x) = 0$

gives $Qx + q^T = 0$

$\Rightarrow Qx = -q^T$

$$\Rightarrow \boxed{x^* = -(Q^{-1} q^T)}$$

1. Set up the GPU

4b

First, make sure your colab has access to a GPU.
Select Runtime -> Change runtime type > GPU.

Check to see if this works by running the following cell.

```
import torch

print(torch.cuda.is_available())
from google.colab import drive
drive.mount('/content/drive') #mounts google drive and it becomes
```

Show hidden output

2. Sync your code to Colab

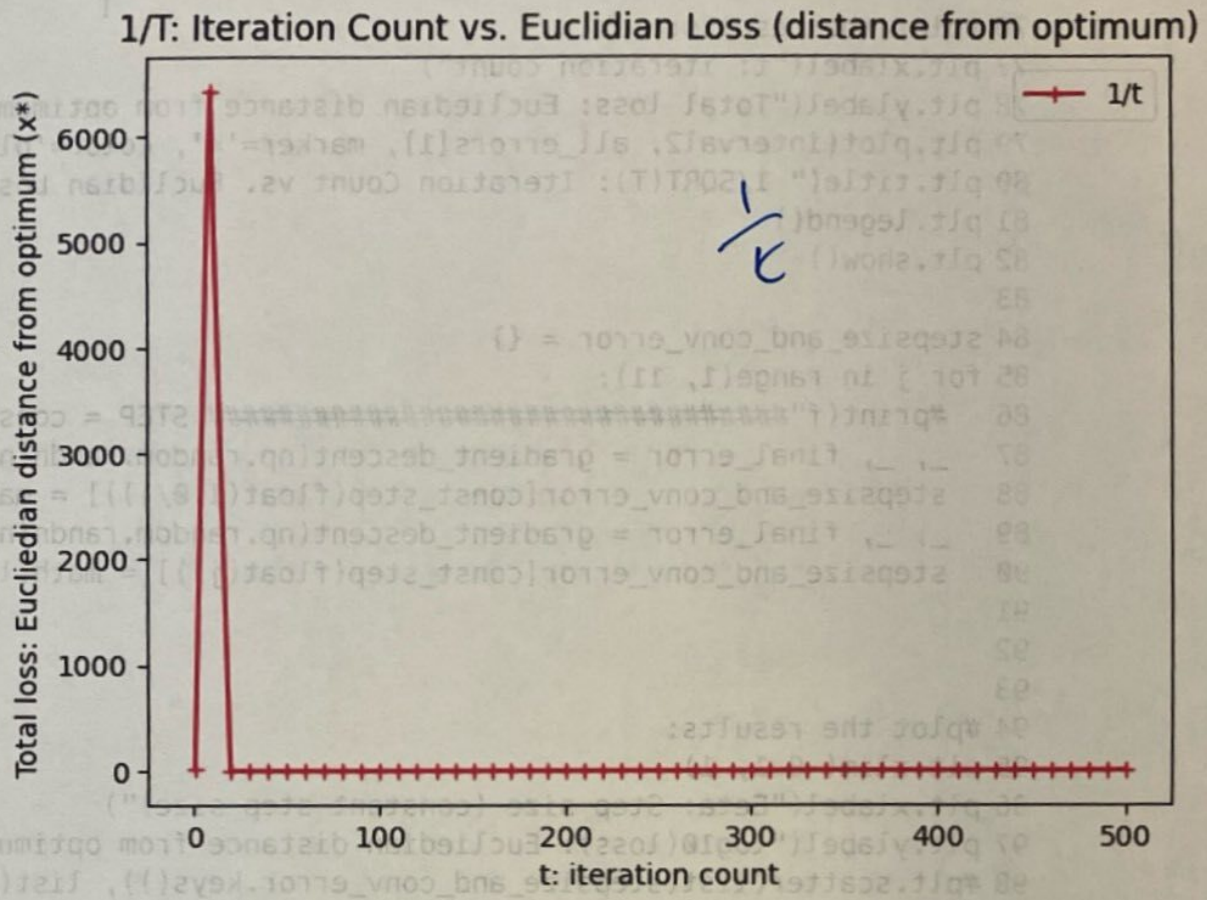
```
import math
from math import sqrt
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings("ignore")

n = 5  #dim: 5D R5
A = np.random.randn(n, n)

Q = A.T @ A + 0.5 * np.eye(n) ##Guarantee that Q is PD
q = np.random.randn(n, 1)
#print(f" Q is {Q} and q is {q}: q_dim {q.shape}")

x_star = -(np.linalg.inv(Q)@q)
#print(f"x_start optimal {x_star} shape {x_star.shape}")

def current_gradient (x, Q, q):
  cg = Q @ x + q
  return cg

def eucliedian_error(x_current):
  return np.linalg.norm(x_current - x_star)

def const_step(multiplier):
```

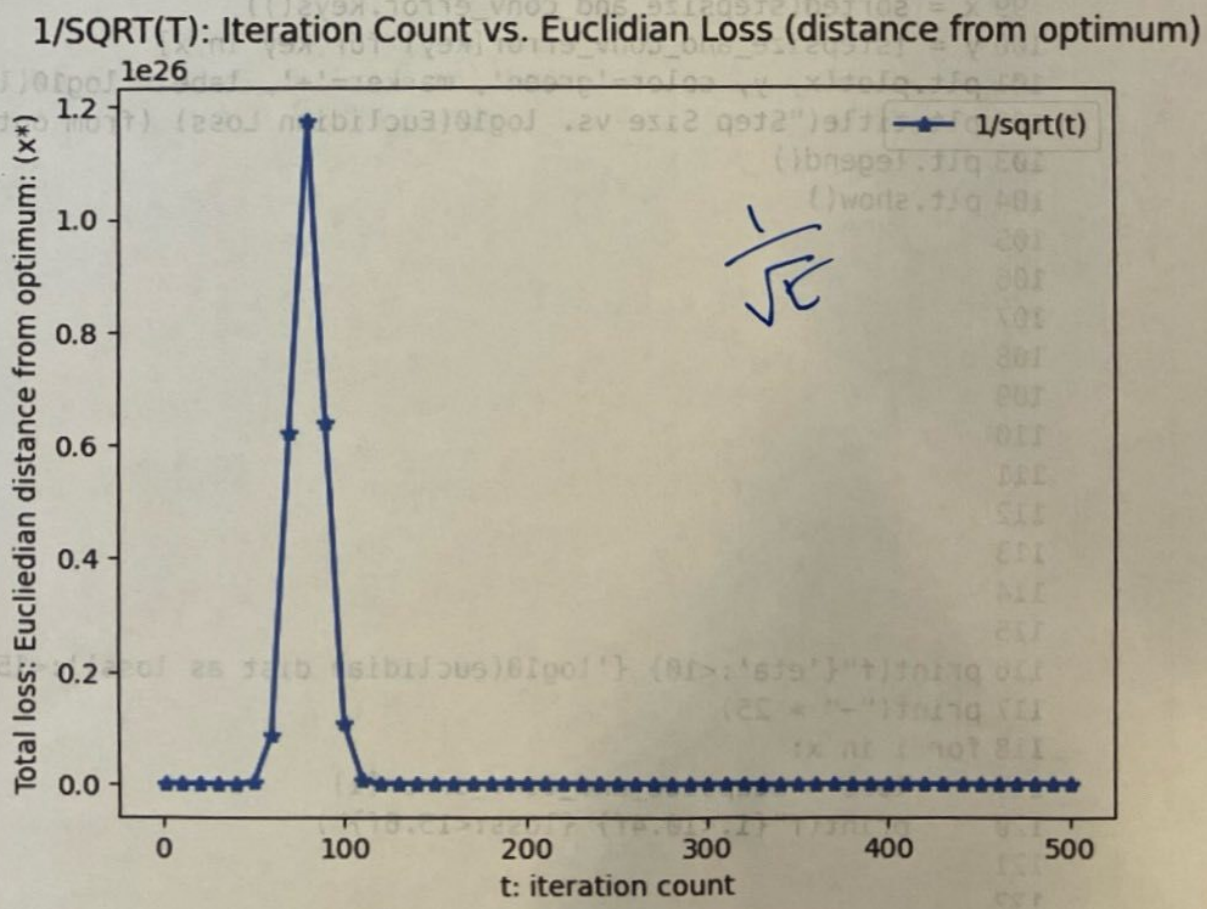
4b

```
  return float(0.1 *multiplier)

def reciprocal_step(t):
  if t == 0:
    return 1.0
  else:
    return (1.0/float(t))

def reciprocal_root_step(t):
  if t == 0:
    return 1.0
  else:
    return (1.0/float(sqrt(t)))

all_errors = []
def gradient_descent(current_x, Q, q, step_fn, num_steps, multipli
  interval = []
  list_error = []
  for t in range(1, num_steps+1):
    cg = current_gradient(current_x, Q, q)
    if multiplier is not None:
      current_x = current_x - step_fn(multiplier) * cg
    else:
      current_x = current_x - step_fn(t) * cg
    current_error = eucliedian_error(current_x)
    if t==1 or t%10 == 0:
      interval.append(t)
      list_error.append(current_error)
    if current_error < 1e-10:
      break
  if np.isnan(current_x).any():
    current_error = 1000 #set some BIG NUMBER
  #print(f"GD converged to solution: {current_x} with error {curre
  return interval, list_error, current_error

#print(f"################################ STEP = 1/t ###########
interval1, euc_error_list, _ = gradient_descent(np.random.randn(n,
all_errors.append(euc_error_list)
#print(f"################################ STEP = 1/sqrt(t) #####
interval2, euc_error_list, _ = gradient_descent(np.random.randn(n,
all_errors.append(euc_error_list)

#plot the results:
plt.xlabel("t: iteration count")
plt.ylabel("Total loss: Eucliedian distance from optimum (x*)")
plt.plot(interval1, all_errors[0], marker='+', color='red', label=
plt.title(" 1/T: Iteration Count vs. Euclidian Loss (distance from
plt.legend()
```

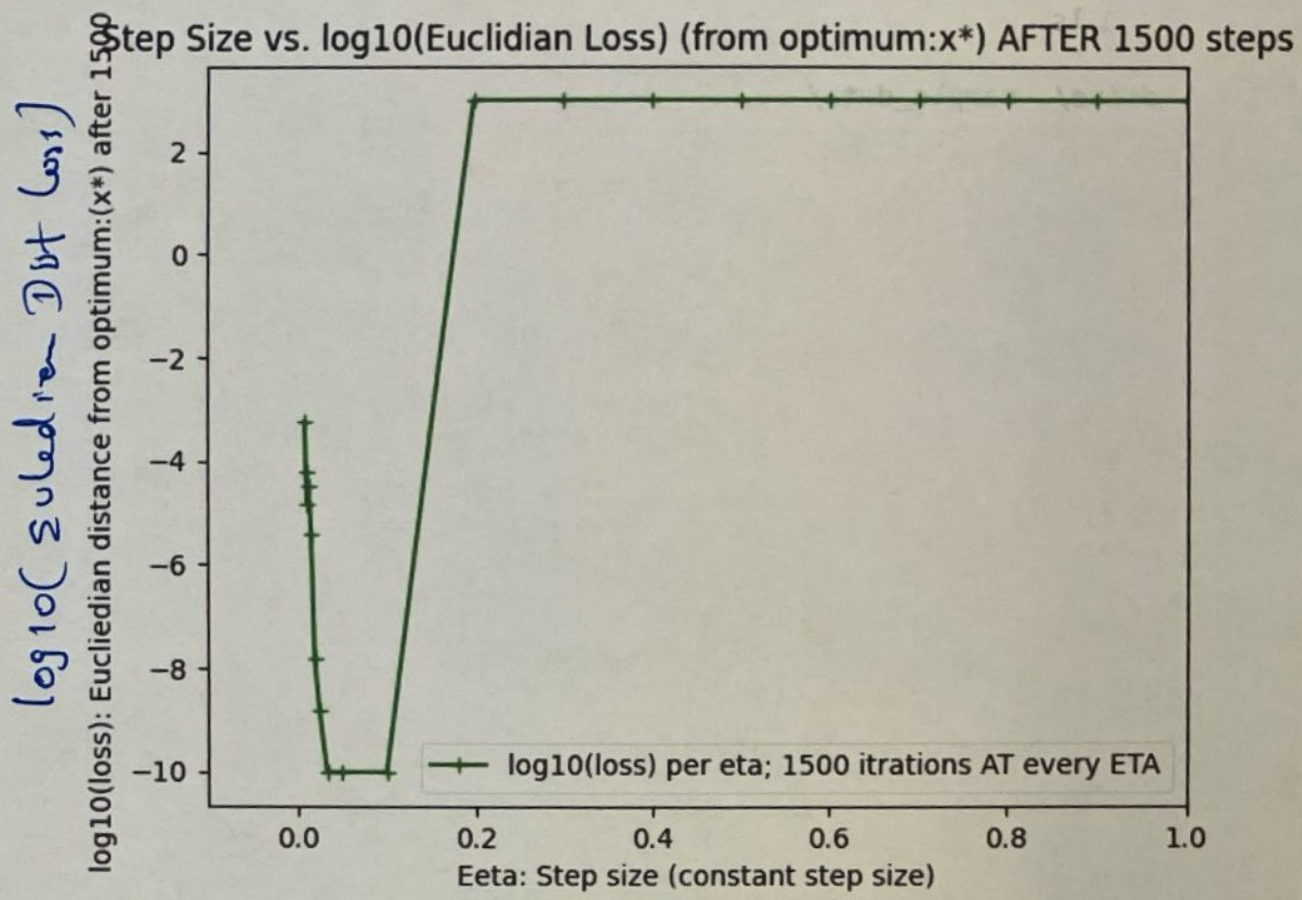
4b

```
plt.show()

#plot the results:
plt.xlabel("t: iteration count")
plt.ylabel("Total loss: Eucliedian distance from optimum: (x*)")
plt.plot(interval2, all_errors[1], marker='*', color='blue', label
plt.title(" 1/SQRT(T): Iteration Count vs. Euclidian Loss (distanc
plt.legend()
plt.show()

stepsize_and_conv_error = {}
for j in range(1, 11):
  #print(f"################################## STEP = const multipl
  _, _, final_error = gradient_descent(np.random.randn(n, 1), Q, c
  stepsize_and_conv_error[const_step(float(1.0/j))] = math.log10(1
  _, _, final_error = gradient_descent(np.random.randn(n, 1), Q, c
  stepsize_and_conv_error[const_step(float(j))] = math.log10(final



#plot the results:
plt.xlim(-0.1, 1)
plt.xlabel("Eeta: Step size (constant step size)")
plt.ylabel("log10(loss): Eucliedian distance from optimum:(x*) aft
#plt.scatter(list(stepsize_and_conv_error.keys()), list(stepsize_a
x = sorted(stepsize_and_conv_error.keys())
y = [stepsize_and_conv_error[key] for key in x]
plt.plot(x, y, color='green', marker='+', label='log10(loss) per e
plt.title("Step Size vs. log10(Euclidian Loss) (from optimum:x*) A
plt.legend()
plt.show()



print(f"{'eta':<10} {'log10(euclidian dist as loss)':<15}")
print("-" * 25)
for i in x:
    loss = stepsize_and_conv_error[i]
    print(f"{i:<10.4f} {loss:<15.6f}")
```


1/T: Iteration Count vs. Euclidian Loss (distance from optimum)
1/t
Total loss: Eucliedian distance from optimum (x*)
t: iteration count
0
1000
2000
3000
4000
5000
6000
0
100
200
300
400
500


1/SQRT(T): Iteration Count vs. Euclidian Loss (distance from optimum)
1e26
1/sqrt(t)
Total loss: Eucliedian distance from optimum: (x*)
t: iteration count
0.0
0.2
0.4
0.6
0.8
1.0
1.2
0
100
200
300
400
500

4C



eta η	log10(euclidian dist as loss)
0.0100	-3.240593
0.0111	-4.219895
0.0125	-4.841763
0.0143	-4.487037
0.0167	-5.400197
0.0200	-7.804790
0.0250	-8.809048
0.0333	-10.005729
0.0500	-10.011071
0.1000	-10.009273
0.2000	3.000000
0.3000	3.000000
0.4000	3.000000
0.5000	3.000000
0.6000	3.000000
0.7000	3.000000
0.8000	3.000000
0.9000	3.000000
1.0000	3.000000

← values of η for which Gradient descent converges

P. 5a

5a) $\min: f_1(x_1) + f_2(x_2) + f_3(x_3) + g(x_1 + x_2 + x_3)$

$$f(x) = f(x_1, x_2, x_3) = f_1(x_1) + f_2(x_2) + f_3(x_3)$$

$$g(Ax) = g(x_1 + x_2 + x_3)$$

$$\text{i.e. } Ax = \underline{x_1 + x_2 + x_3} = [1 \;\; 1 \;\; 1]\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

$$A^T = [1 \;\; 1 \;\; 1] \qquad x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

$$\min: \; f(x) + g(Ax)$$

$$\min: \; f(x) + g(y)$$

$$\text{s.t. } Ax = y$$

writing the Dual $\Rightarrow$

$$\min_{x,y} \max_{z}: \; f(x) + g(y) + z^T(Ax - y)$$

$$\max_{z} \min_{x,y}: \; (A^T z)^T x + f(x) + g(y) - z^T y$$

$$\max_{z} \min_{x}: \; -[(-A^T z)^T x - f(x)] + \min_{y} -(z^T y - g(y))$$

$$\max_{z} \; -\max_{x}: \; ((-A^T z)^T x - f(x)) - \max_{y}: \; (z^T y - g(y))$$

$$\max_{z} \; - f^*(-A^T z) - g^*(z)$$

P. 59

$$\max_{z} \; -f^*(-A^T z) - g^*(z)$$

$$A = [1 \;\; 1 \;\; 1]$$

$$\Rightarrow \max_{z} \; -f_1^*(-z) - f_2^*(-z) - f_3^*(-z) - g^*(z)$$

This is a univariate optimization problem

Q. 5b

5b) Let the dual of min $f_1(x_1)+f_2(x_2)+f_3(x_3)+g(x_1+x_2+x_3)$ as derived in 5(a) be $\max_z h(z)$

$$h(z) = -f_1^*(-z) - f_2^*(-z) - f_3^*(-z) - g^*(z)$$

The gradient update step for z,

$$z_{t+1} = z_t + \eta_t \nabla h(z_t)$$

(as this is maximization problem, we move in the direction of gradient)

$$f^*(y) = \sup_x (yx - f(x))$$

$$f^*(y) = yx - f(x)$$

$$\frac{\partial}{dy} f^*(y) = x - 0 = x^*$$

$$\boxed{\frac{d}{dy} f^*(y) = x^*(y)}$$

From this $\frac{d}{dz}(-f_1^*(-z)) = -\left(\frac{df_1^*}{d(-z)} \cdot \frac{d(-z)}{dz}\right) = -(x_1 \cdot -1) = x_1$

Similarly $\frac{d}{dz}(-f_2^*(-z)) = x_2$

$\frac{d}{dz}(-f_3(-z)) = x_3$

Q. 5b

$$\frac{d}{dz}(-g^*(z)) = -\frac{d}{dz}(g^*(z)) = -Ax$$

$$\nabla h(z) = x_1(z) + x_2(z) + x_3(z) - Ax(z)$$

Step size

$$z_{t+1} = z_t + \eta_t(x_1 + x_2 + x_3 - \underline{Ax})_t$$

notice that Ax as defined in 5(a)

$$Ax = (x_1 + x_2 + \underline{x_3})$$